

Lezione 16: Gestione degli Errori tramite Eccezioni (Exceptions)

Questo notebook raccoglie tutto il codice della Lezione 16 (MavenDate):

- `src/main/java/it/oop/exception/IllegalDateException.java` (Eccezione Unchecked)
- `src/main/java/it/oop/exception/OrderdPairException.java` (Eccezione Checked)
- Classi di supporto: `Date` , `FormattedDate` , `ItalianDate` , `AmericanDate` , `OrderedPair` , `DateInterval` , `Pair` , `DatePair`
- `src/main/java/it/oop/ui/MainDate.java`
- `src/test/java/it/oop/core/TestItalianDate.java`

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/
├── codice-commentato/
│   └── Lezione16/
│       ├── MavenDate
│       │   ├── pom.xml
│       │   └── src
│       │       ├── main
│       │       │   ├── java
│       │       │   │   └── it
│       │       │   │       └── oop
│       │       │   │           ├── core
│       │       │   │           │   ├── AmericanDate.java
│       │       │   │           │   ├── BirthDay.java
│       │       │   │           │   ├── Date.java
│       │       │   │           │   ├── DateInterval.java
│       │       │   │           │   ├── DatePair.java
│       │       │   │           │   ├── FormattedDate.java
│       │       │   │           │   ├── FormattedDateConverter.java
│       │       │   │           │   ├── ItalianDate.java
│       │       │   │           │   ├── OrderedPair.java
│       │       │   │           │   ├── Pair.java
│       │       │   │           │   ├── Time.java
│       │       │   │           │   └── TimeStamp.java
│       │       │   │           └── exception
│       │       │   │               ├── IllegalDateException.java
│       │       │   │               └── OrderdPairException.java
│       │       │   │           └── ui
│       │       │   │               ├── Date.java
│       │       │   │               └── MainDate.java
│       │       └── resources
│       └── test
│           ├── java
│           │   └── it
│           │       └── oop
│           │           └── core
│           │               └── TestItalianDate.java
```

Argomenti trattati:

- Gerarchia delle eccezioni in Java: `Throwable` -> `Exception` e `RuntimeException`
- Differenza tra eccezioni controllate (Checked, che estendono `Exception` e richiedono `try-catch` o clausola `throws`) ed eccezioni non controllate (Unchecked, che estendono `RuntimeException`)
- Creazione di eccezioni custom: `IllegalDateException` e `OrderdPairException`
- Gestione robusta dell'input utente con ciclo `while` e recupero da `InputMismatchException`
- Clausole `try` , `catch` , `finally` e propagazione delle eccezioni

1. Eccezioni Custom: `IllegalDateException` e `OrderdPairException`

```
In [1]: // Eccezione non controllata (Unchecked RuntimeException)
class IllegalDateException extends RuntimeException {
    public IllegalDateException(String message) {
        super(message);
    }
}

// Eccezione controllata (Checked Exception)
class OrderPairException extends Exception {
    public OrderPairException(String message) {
        super(message);
    }
}
```

2. Classe `Date` con lancio di `IllegalDateException`

```
In [2]: import java.util.Objects;

class Date implements Comparable<Date> {
    protected int day;
    protected int month;
    protected int year;

    public Date(int day, int month, int year) {
        this.day = day;
        this.month = month;
        this.year = year;
        verify();
    }
    public Date(int day, int month) {
        this(day, month, 2025);
    }
    public Date(Date other) {
        this(other.day, other.month, other.year);
    }

    void verify() {
        if (year < 0)
            throw new IllegalDateException("Illegal date: wrong year");
        if (month < 1 || month > 12)
            throw new IllegalDateException("Illegal date: wrong month");
        if (day < 1 || day > daysPerMonth(month))
            throw new IllegalDateException("Illegal date: wrong day");
    }

    public int getDay() { return day; }
    public int getMonth() { return month; }
    public int getYear() { return year; }

    public static int daysPerMonth(int month) {
        int days;
        switch(month) {
            case 4: case 6: case 9: case 11:
                days = 30; break;
            case 2:
                days = 28; break;
            default:
                days = 31; break;
        }
        return days;
    }

    @Override
    public String toString() {
        return String.format("%dm%dd%d", year, month, day);
    }

    @Override
    public boolean equals(Object other) {
        if (other == null) return false;
        if (this == other) return true;
        if (!(other instanceof Date)) return false;
    }
}
```

```

        Date otherAsDate = (Date) other;
        return this.day == otherAsDate.getDay() &&
               this.month == otherAsDate.getMonth() &&
               this.year == otherAsDate.getYear();
    }

    @Override
    public int hashCode() {
        return Objects.hash(day, month, year);
    }

    @Override
    public int compareTo(Date otherAsDate) {
        int diff = this.year - otherAsDate.getYear();
        if (diff != 0) return diff;
        diff = this.month - otherAsDate.getMonth();
        if (diff != 0) return diff;
        return this.day - otherAsDate.getDay();
    }

    public static class Builder {
        private final int year;
        public Builder(int year) {
            this.year = (year > 0) ? year : 1970;
        }
        public Date build(int day, int month) {
            if (month < 1 || month > 12 || day < 1 || day > daysPerMonth(month))
                return new Date(1, 1, year);
            return new Date(day, month, year);
        }
    }
}

```

3. Classi `FormattedDate` e `ItalianDate`

```

In [3]: abstract class FormattedDate extends Date {
        protected final String format;
        protected final String[] months;

        public FormattedDate(int day, int month, int year, String format, String[] months) {
            super(day, month, year);
            this.format = format;
            this.months = months;
        }

        public final String printFormat() { return format; }
        public final String getMonthAsString() { return months[getMonth()-1]; }
        public abstract String prettyPrint();
    }

    class ItalianDate extends FormattedDate {
        private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile", "maggio",
        "giugno", "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre" };
        public ItalianDate(int day, int month, int year) {
            super(day, month, year, "dd/mm/yyyy", MONTHS_IT);
        }
        @Override
        public String prettyPrint() { return day + " " + getMonthAsString() + " " + getYear(); }
        @Override
        public String toString() { return day + "/" + getMonth() + "/" + getYear(); }
    }
}

```

4. `OrderedPair` e `DateInterval` con clausola `throws OrderdPairException`

```

In [4]: class OrderedPair<T> extends Comparable<T> {
        private final T first;
        private final T second;

        public OrderedPair(T first, T second) throws OrderdPairException {
            this.first = first;
            this.second = second;
        }
    }

```

```

        if (first.compareTo(second) > 0)
            throw new OrderdPairException("First element must be less or equal than second one.");
    }

    public T getFirst() { return first; }
    public T getSecond() { return second; }
}

class DateInterval extends OrderedPair<Date> {
    public DateInterval(Date left, Date right) throws OrderdPairException {
        super(left, right);
    }
    public Date getLeft() { return getFirst(); }
    public Date getRight() { return getSecond(); }
    @Override
    public String toString() {
        return String.format("[%s .. %s]", getLeft().toString(), getRight().toString());
    }
}

```

5. Classe `MainDate` (Gestione delle Eccezioni) ed Esecuzione

```

In [5]: import java.util.InputMismatchException;
import java.util.Scanner;
import java.io.ByteArrayInputStream;

class MainDate {
    public static void main(String[] args) {
        System.out.println("Insert day, month, year as numbers"); // Insert day, month, year as numbers
        // Hardcoding intelligente dell'input da tastiera (simulazione stream System.in con i valori 15,
1, 2025)
        System.setIn(new ByteArrayInputStream("15 1 2025\n".getBytes()));
        int d = -1, m = -1, y = -1;
        boolean correct = false;
        while (correct == false) {
            try {
                Scanner sc = new Scanner(System.in);
                d = sc.nextInt();
                m = sc.nextInt();
                y = sc.nextInt();
                correct = true;
            } catch (InputMismatchException ime) {
                System.out.println("Input not valid, retry"); // (eseguito in caso di input non numerico
da tastiera)
            }
        }
        try {
            FormattedDate date = new ItalianDate(d, m, y);
            System.out.println(date.toString()); // 15/1/2025
        } catch (IllegalDateException ide) {
            System.out.println("Date not valid: " + ide.getMessage()); // (non eseguito: la data
15/1/2025 è valida)
        }

        try {
            // Tentativo di creare un intervallo non ordinato (31/1/2025 > 1/1/2025)
            DateInterval di = new DateInterval(new Date(31, 1, 2025), new Date(1, 1, 2025));
        } catch (OrderdPairException ope) {
            System.out.println("Eccezione catturata con successo: " + ope.getMessage()); // Eccezione
catturata con successo: First element must be less or equal than second one.
            ope.printStackTrace(); // it.oop.exception.OrderdPairException: First element must be less or
equal than second one.
        }
    }
}
MainDate.main(null);

```

Insert day, month, year as numbers
15/1/2025

Eccezione catturata con successo: First element must be less or equal than second one.

```
REPL.$JShell$3$OrderedPairException: First element must be less or equal than second one.
    at REPL.$JShell$8$OrderedPair.<init>($JShell$8.java:22)
    at REPL.$JShell$9$DateInterval.<init>($JShell$9.java:16)
    at REPL.$JShell$13$MainDate.main($JShell$13.java:41)
    at REPL.$JShell$14.do_it($JShell$14.java:14)
    at
java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:104)
    at java.base/java.lang.reflect.Method.invoke(Method.java:565)
    at
org.rapaio.jupyter.kernel.core.java.RapaioExecutionControl.lambda$execute$0(RapaioExecutionControl.java:58
)
    at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:328)
    at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1090)
    at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:614)
    at java.base/java.lang.Thread.run(Thread.java:1474)
```